

`f x = <kroppen> - - hvor kroppen kaller f ("f kaller seg selv")`

```
app l1 l2 =
  if null(l1) then l2 else (head l1):app (tail l1) l2
```

```
appm [] ls = ls
appm (x:xs) ls = x : appm xs ls
```

```
rev [] = []
rev (x:xs) = (rev xs) ++ [x]
```

```
rev [] akk = akk
rev (x:xs) akk = rev xs (x:akk)
```

```
qs [] = []
qs (x:xs) = qs[y|y<-xs,y<=x] ++ [x] ++ qs[y|y<-xs,y>x]
```

Gitt en instans n av et problem P :

1. Hva gjør jeg når n er basistilfellet
– for lister $n = []$
2. Hvordan konstruere løsning for $P(n)$ utfra løsninger $P(m_i)$ for mindre instanser $m_i < n$?
– for lister: hvordan konstruere løsning for $P(x : xs)$ fra løsning for $P(xs)$.

1. Hvilke datatype brukes for rekursiv løsning av et problem P ?

INDUKTIVE...

2. Induktive Data Typer *(vilkårlig store men endelige)*

naturlige tall N:

basis: **0** er et N

hvis **n** er et N

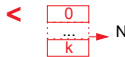
så er: **n+1** et N

array av N: A(N)

basis **0** $\rightarrow$ **N** er A(N)

hvis **[0...k]** $\rightarrow$ **N** er A(N)

så er **[0...k,k+1]** $\rightarrow$ **N** er A(N)

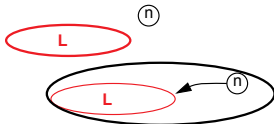


Lister av N: L(N):

basis: **null** er en L(N)

hvis **L** er L(N) og **n** er N

så er: **(n,L)** en L(N)

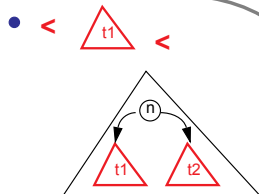
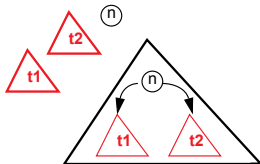


Binære Trær av N: BT(N):

basis: **null** er et BT(N)

hvis **t1, t2** er BT(N) og **n** er N

så er: **(t1, n, t2)** et BT(N)



"Strukturell ordering"

Fremgangsmåten: “splitt og hersk” på induktive DT

Gitt en instans n av P :

1. Hva gjør jeg når n er basistilfellet?
2. Hvordan konstruere løsning $P(n)$ utfra løsninger $P(m_i)$ for mindre instanser $m_i < n$?

► $P(n) = n!$

► Rekursjon over n (naturlige tall)

► basis, $n = 0$, verdien: $P\ 0 = 1$

► induksjonssteget, +1:

$$P\ n = P\ (n - 1) * n$$

Fremgangsmåten: “splitt og hersk” på induktive DT

Gitt en instans n av P :

1. Hva gjør jeg når n er basistilfellet?
 2. Hvordan konstruere løsning $P(n)$ utfra løsninger $P(m_i)$ for mindre instanser $m_i < n$?
- ▶ $bs\ x\ l =$ finn el. med nøkkel x i en nøkkel-sortert liste l
 - ▶ Rekursjon over lengden av listen (naturlige tall)
 - ▶ basis, $[]$ eller $[x]$, verdien:
 $bs\ x\ [] = \text{error "Ingen"}$
 $bs\ x\ [(y,z)] \mid x==y = z$
 $\mid \text{otherwise} = \text{error "Ingen"}$
 - ▶ induksjonssteget, halver listen:
 $bs\ x\ l = \text{let } m = \text{lenght } l \text{ 'div' } 2, (n,v) = (l !! m) \text{ in}$
 if $n==x$ then v
 else if $n < x$ then $bs\ x\ (\text{drop } m\ l)$
 else $bs\ x\ (\text{take } m\ l)$

Fremgangsmåten: “splitt og hersk” på induktive DT

Gitt en instans n av P :

1. Hva gjør jeg når n er basistilfellet?
 2. Hvordan konstruere løsning $P(n)$ utfra løsninger $P(m_i)$ for mindre instanser $m_i < n$?
- ▶ $P(n)$ = summer tall i et binært tre
 - med n elementer?
 - av høyde n ?
 - ▶ data BT = BI Int | Nd BT Int BT
 - ▶ Rekursjon over “kompleksiteten” av treet (induktiv def.)
 - ▶ basis : sum (BI x) = x
 - ▶ induksjonssteget:
sum (Nd vt x ht) = x + sum vt + sum ht

variasjoner:

Lineær rekursjon

f x = <kroppen> hvor f kaller seg selv én gang

```
snu l1 =  
  if null(l1) then l1 else snu(tail l1)++head(l1)
```

```
app l1 l2 =  
  if null(l1) then l2 else head l1:app (tail l1) l2
```

```
rev l1 l2 =  
  if null(l1) then l2 else rev (tail l1) (head l1:l2)
```

variasjoner:

Ikke linær rekursjon (flere rekursive kall)

sum Nd vt x ht = x + sum vt + sum ht

qs [] = []

qs (x:xs) = qs [y | y <- xs, y <= x] ++ [x] ++ qs [y | y <- xs, y > x]

fib 0 = 1

fib 1 = 1

fib n = fib(n-1) + fib(n-2)

Flere argumenter: rek. men i **hvilket argument** (hvis ikke begge)?

zip [] _ = []

zip _ [] = []

zip (x:xs) (y:ys) = (x,y) : zip xs ys

$\text{exp } x \ n = \underbrace{x * \dots * x}_n?$

$\text{exp } x \ 0 = 1$

$\text{exp } x \ n = x * (\text{exp } x \ (n-1))$

variasjoner: gjensidig rekursjon

- ▶ $\text{even } 0 = \text{True}$
 $\text{even } n = \text{odd } n-1$
 $\text{odd } 0 = \text{False}$
 $\text{odd } n = \text{even } n-1$
 $\text{odd } 2 = \text{even } 1 = \text{odd } 0 = \text{False}$
- ▶ del liste i to: én med odde og andre med like indekser
 $\text{di } [0,1,2,3] = ([0,2],[1,3])$ $\text{di } [0] = ([0],[])$
- $\text{di } \text{xs} = (\text{li } \text{xs}, \text{uli } \text{xs})$

$\text{li } [] = []$	$\text{uli } [] = []$
$\text{li } [x] = [x]$	$\text{uli } [x] = []$
$\text{li } (x:y:\text{xs}) = x : \text{li } \text{xs}$	$\text{uli } (x:y:\text{xs}) = y : \text{uli } \text{xs}$
- ▶ $\text{di } [] = ([], [])$
 $\text{di } (x:\text{xs}) = \text{let } (a,b) = \text{di2 } \text{xs} \text{ in } (x:a,b)$
 $\text{di2 } [] = ([], [])$
 $\text{di2 } (x:\text{xs}) = \text{let } (a,b) = \text{di } \text{xs} \text{ in } (a,x:b)$

Generell rekursjon implementeres med en stabel...

Lister: konstruksjon: **[]**, **:** ,

destruksjon: **head**, **tail**,

basistilfelletest: **null**

Imperativ “reverse” av liste l1:

```
list l1 = ...;
```

```
list l2 = [];
```

```
while (! null(l1)) l2 = head(l1):l2; l1 = tail(l1);
```

Imperativ “append” av l2 etter l1:

```
list l1 = ...;
```

```
list l2 = ...;
```

```
list l3 = [];
```

```
while (! null(l1)) l3 = head(l1):l3; l1 = tail(l1);
```

```
while (! null(l3)) l2 = head(l3):l2; l3 = tail(l3);
```

Imperativ “append” av l2 etter l1:

		c:
	d:	b:
	e:	a:
[]	[]	[]
—	—	—
l1	l2	l3

```
while (! null(l1)) l3 = head(l1):l3; l1 = tail(l1);  
while (! null(l3)) l2 = head(l3):l2; l3 = tail(l3);
```

Vi trenger en hjelpeliste l3 og to løkker (winding/unwinding).

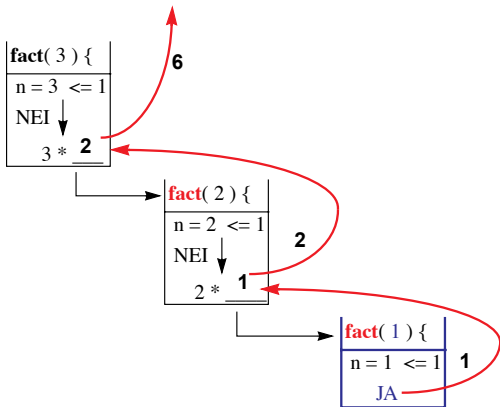
Til en generell implementasjon av rekursjon trenges en stabel.

`app [] bs = bs` `app (a:as) bs = a : app as bs`

`app [1,2,3] [5] = 1 : (app [2,3] [5]) = 1 : 2 : (app [3] [5])`
`= 1 : 2 : 3 : (app [] [5]) = 1 : 2 : 3 : [5]`

4a. Rekursjon implementeres med en stabel

```
public int fact( int n ) {  
    if ( n <= 1 ) return 1;  
    else return n * fact( n-1 );  
}
```

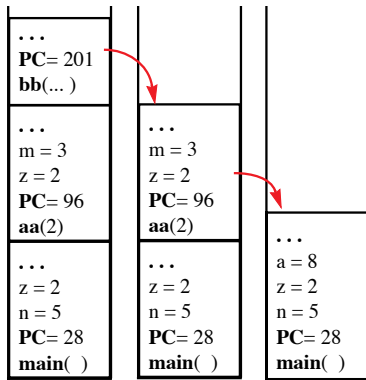
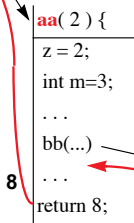
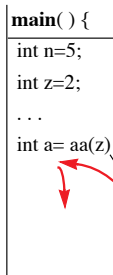


'Method Stack'

```
main() {  
    int n=5;  
    int z=2;  
    ...  
28   int a= aa(z);  
    ...  
}
```

```
int aa(int z) {  
    int m=3;  
    ...  
96   bb(...);  
    ...  
    return 8;  
}
```

```
201 void bb(...) {  
    ...  
}
```

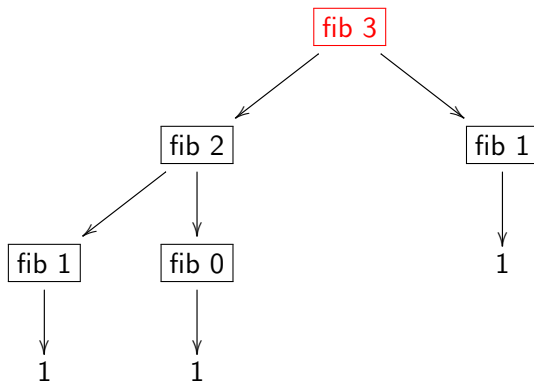


Rekursjonstre

fib 0 = 1

fib 1 = 1

fib n = fib (n-1) + fib (n-2)



Lineær rekursjon – kan være halerekursjon

`f x = <kroppen>` - - hvor `f` kaller seg selv én gang
som siste aksjon i funksjonskroppen

```
app l1 l2 =  
  if null(l1) then l2 else head l1:app (tail l1) l2
```

```
snu l1 =  
  if null(l1) then l1 else snu(tail l1)++(head l1)
```

```
rev l1 l2 =  
  if null(l1) then l2 else rev (tail l1) (head l1:l2)
```

Kun `h-!C!C[u]rev` er halerekursiv (tail recursive)

Halerekursjon

- ▶ Vi trenger **ikke** en hjelpeliste for imperativ “reverse”.

```
list l1 = ...; list l2 = [];
```

```
while (! null(l1))
```

```
    l2 = head(l1):l2; l1 = tail(l1);
```

fordi “reverse” er (kan skrives som) halerekursjon.

```
rev [] acc = acc
```

```
rev (x:xs) acc = rev xs (x:acc)
```

```
rev [1,2,3] [] = rev [2,3] [1] = rev [3] [2,1] =      [3,2,1]
```

Trenger kun én ekstra output variabel (akkumulator acc).

- ▶ Halerekursjon trenger ikke en stabel, fordi tilstand (aktuelle variabelverdier) overføres til neste kall trivielt (oppdatering)
 - Dermed bruker halerekursjon atskillig mindre minne.
 - og kan lett (mekanisk) omskrives til iterasjon.
 - Halerekursjon-optimering er obligatorisk for FP kompilering!

Iterasjon	til rekursjon	m. akkumulator
<pre> sum(n) = r:= 0; while (n > 0) r:= r+n; n:= n-1; return r; </pre>	<pre> sum 0 = 0 sum n = n + sum(n-1) </pre>	<pre> sum n = sm n 0 sm 0 r = r sm n r = sm n-1 r+n halerekursjon! </pre>

Enhver iterasjon kan skrives som rekursjon, tom. som **halerekursjon** ved bruk av ekstra akkumulator parameter.

<pre> f(n) = r:= init; while (n > sist) r:= kropp(r,n); n:= steg(r,n); return r; </pre>	<pre> f n = fu n init fu sist r = r fu n r = fu steg(r,n) kropp(r,n) </pre>
--	---

Halerekursjon – kan øke tiden eller vanskelighetsgraden

??? $\text{append } [] \text{ bs} = \text{bs}$ $\text{fact } 1 = 1$
 $\text{append } (a:\text{as}) \text{ bs} = a : \text{append as bs}$ $\text{fact } n = n * \text{fact}(n-1)$

1. $\text{app as bs} = \text{apph (reverse as) bs}$ $\text{fact } n = \text{aux } n \ 1$
 $\text{apph } [] \text{ bs} = \text{bs}$ $\text{aux } 1 \ x = x$
 $\text{apph } (x:\text{xs}) \text{ bs} = \text{apph xs } (x:\text{bs})$ $\text{aux } n \ x = \text{aux } (n-1) \ (n * x)$

2. eksplisitt akkumulator:
 $\text{append as bs} = \text{appa as } [] \text{ bs}$
 $\text{appa } (a:\text{as}) \text{ akk bs} = \text{appa as } (a:\text{akk}) \text{ bs}$
 $\text{appa } [] \ (a:\text{akk}) \text{ bs} = \text{appa } [] \ \text{akk } (a:\text{bs})$
 $\text{appa } [] \ [] \text{ bs} = \text{bs}$

!! Generelt: halerekursjon anbefales.

Rekursjon – oppsummering

- ▶ “Splitt og hersk”:

 - basis:** bestem hva som skal gjøres med de(t) minste tilfelle(r)

 - splitt:** bruk rekursive løsninger for mindre instanser

 - hersk:** for å konstruere en løsning for den aktuelle instansen

- ▶ Terminering

 - hvert rekursivt kall må bringe oss nærmere basistilfelle

- ▶ Enhver induktiv datatype gir opphav til rekursive algoritmer.

- ▶ Generell rekursjon implementers iterativt med stabel

 - halerekursjon kan implementeres uten (essensielt ved FP)

- ▶ Hutton, kap. 6 (spesielt, 6.6!).